
Introduction to Agile

**AN INTRODUCTION TO AGILE WAYS OF
WORKING FOR GOVERNMENT TEAMS**

Version: 1.0

Last updated: March 27, 2025

Digital Transformation Team

Office of the Chief Information Officer

Finance and Treasury Board

Contents

Introduction.....	3
What's inside this guide	3
Who this guide is for.....	3
What is Agile?	4
Agile is a way of working.....	4
A brief history of Agile	4
The shift from Waterfall to Agile	4
Why this shift matters to government	5
Agile values and principles.....	6
Agile values	6
Agile principles	7
Key Agile concepts.....	16
Popular Agile frameworks and approaches	18
Scrum.....	18
Kanban.....	20
Extreme Programming (XP).....	22
Other Agile frameworks	23
The Digital Transformation Team's Human-Centered Agile Framework	25
Get started with Agile.....	29

Introduction

What's inside this guide

This guide introduces Agile values, principles, and ways of working for government context. It is not a step-by-step manual for applying Agile, as Agile is not a one-size-fits-all approach. Instead, this guide provides an overview of key concepts to help teams understand Agile and determine how to best apply its values and principles in their work.

This guide will introduce you to:

- ✓ The values and principles of Agile
- ✓ Different frameworks and approaches
- ✓ The Digital Transformation Team's Human-Centered Agile Framework
- ✓ Practical steps to get started with Agile

Who this guide is for

This introduction to Agile is for anyone who wants to improve the way their team works and delivers value. Whether you are a project manager, policy advisor, business analyst, designer, developer or executive leader, Agile principles can help break down silos, improve teamwork, and enable faster, more effective decision-making.

This guide is for:

- ✓ Anyone looking for an introduction to key Agile concepts
- ✓ Teams considering adopting Agile ways of working
- ✓ Leaders and decision-makers who want to support Agile ways of working

What is Agile?

Agile is a way of working

Agile is not a single process, framework, or methodology — it's a way of working that prioritizes collaboration, transparency, and responsiveness to change. Any approach that aligns with the [Agile Manifesto's values](#) and [principles](#) can be considered Agile. Rather than prescribing rigid steps, Agile provides a flexible foundation for teams to navigate uncertainty, adapt, and continuously improve their work

A brief history of Agile

For years, software development followed a rigid, linear approach: define all requirements upfront, build everything according to the plan, and deliver the final product months or even years later. This method, known as the Waterfall approach, assumes that project needs won't change along the way. In reality, they almost always do. As a result, projects often ran over budget, took too long to complete, or failed to meet expectations. By the time a product was delivered, user needs or technology had already evolved.

Throughout the 1990s, software developers explored new ways of working — looking for methods that embraced change instead of resisting it. This led to frameworks like Extreme Programming, Scrum, Feature-Driven Development, and Crystal Clear, all built on shared principles of flexibility, collaboration, and continuous learning.

In 2001, a group of developers formalized these ideas in the Agile Manifesto, defining the values and principles that shape Agile today. Their work captured the philosophy behind these frameworks and transformed not just software development but many other fields.

The shift from Waterfall to Agile

As organizations recognized the limitations of the Waterfall approach, many began adopting Agile methods to improve efficiency and responsiveness. Unlike Waterfall, which relies on detailed upfront planning, Agile breaks work into smaller, iterative cycles, allowing teams to adapt as they go. This shift enabled faster feedback loops, continuous learning, and a greater focus on delivering value early and often.

The transition required a fundamental change in mindset. Teams had to rethink how they collaborated, moving away from rigid roles and silos toward cross-functional teamwork and shared accountability. Leaders also had to shift their approach, embracing flexibility, trust, and iterative progress rather than detailed control.

Over time, Agile's benefits led to widespread adoption, extending beyond software development into fields like marketing, operations, and even government service delivery.

Why this shift matters to government

Government operates in a complex, ever-changing environment influenced by evolving policies, emerging technologies, and shifting public needs. Agile provides a way to navigate this uncertainty by enabling teams to work iteratively, adapt to change, and continuously improve their approach.

By adopting Agile, governments move toward more collaborative, responsive ways of working. Agile encourages cross-functional teams to break down silos, test ideas in real-world conditions, and adjust based on evidence rather than assumptions. It helps public servants focus on delivering value faster — whether by improving internal processes, refining public services, or responding more effectively to urgent challenges. As expectations for government service delivery continues to evolve, Agile offers frameworks for working in ways that are transparent, accountable, and better aligned with the needs of the people it serves.

A New Brunswick road trip analogy: Imagine you're driving from Edmundston to Moncton...

Waterfall: You meticulously plan every kilometre in advance. You decide exactly when and where to stop, where to eat, and what time you'll arrive. But then, halfway through, there's a snowstorm in Fredericton. Your plan doesn't allow for changes or delays, so you either keep driving into unsafe conditions or start your trip over.

Agile: You plan your destination and general timeline, but you stay flexible. You check the forecast and adjust your route in real-time. If you hear there's black ice, you take a different road. If an unexpected stop in Fredericton makes sense, you adapt. The goal is the same — getting to Moncton safely — but you're adjusting along the way to make the best decisions.



Agile values and principles

At its core, Agile is about mindset and behavior. While there are many Agile frameworks, they all share a common foundation: the Agile Manifesto and its principles. These guide how teams approach their work, emphasizing adaptability, collaboration, and a focus on delivering value.

Agile values

The [Agile Manifesto](#) focuses on four key values:

- 1. Individuals and interactions over processes and tools.**
- 2. Working solutions over comprehensive documentation.**
- 3. Customer collaboration over contract negotiation.**
- 4. Responding to change over following a plan.**

These values represent a major shift from traditional, process-heavy approaches. However, Agile doesn't dismiss the items on the right — it simply prioritizes the left. That means processes, tools, documentation, contracts, and planning still have a place, but they shouldn't get in the way of what truly drives success: people, collaboration, adaptability, and delivering real value.

Agile principles

Building on these values, the Agile Manifesto defines [12 principles](#). These principles help teams navigate uncertainty, stay focused on outcomes, and continuously improve how they work. While they were originally created for software development, the underlying ideas can be applied across many types of work in government. They support more responsive policies, higher-quality services, and stronger accountability to the public.

Let's explore each principle, along with a short explanation of what it means and how it might show up in practice — even if you're not developing software.

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

What it means

Deliver useful, working solutions in small increments rather than waiting for a full launch. Prioritize progress over perfection to meet real needs sooner, learn what works, and adjust based on feedback.

The principle in practice

- ✔ **Online driver's license renewal:** Instead of waiting to redesign the entire licensing system, a provincial government rolls out a basic online renewal service first. Users can renew existing licenses online while other features like booking road tests and updating addresses are developed and added based on usage data and feedback.
- ✔ **Emergency relief programs (e.g. COVID-19 benefits):** Governments rolled out initial pandemic-related financial relief programs quickly, often with minimal eligibility checks or automation. Over time, they added more features — like direct deposit, self-service portals, and additional benefit types — based on user needs and operational capacity.
- ✔ **Public parks and facilities:** Instead of waiting years for full park redesigns, a local government runs quick wins like temporary play structures, pop-up seating, and shaded areas. These are monitored and refined through community input before permanent installations are planned.

2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

What it means

Don't treat plans as fixed. Stay open to feedback, new information, or shifting needs. Flexibility helps deliver solutions that remain relevant and valuable.

The principle in practice

- ✔ **Public health portal updates:** Midway through building a vaccine booking system, a provincial government shifts gears to add family booking functionality after realizing many parents are trying to schedule for multiple people. The team reprioritizes features and integrates the change, improving uptake and user satisfaction.
- ✔ **Immigration application tool:** During development, user research reveals that newcomers are struggling with jargon in form labels. Even though the content team had finalized copy, they rewrite and simplify language late in the sprint — prioritizing clarity based on real-world usage.
- ✔ **Childcare subsidy program:** After launching a subsidy program, feedback from applicants and community organizations reveals gaps in eligibility for gig workers. The policy team adapts mid-rollout, changing criteria and reworking communications to be more inclusive.

3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter timescale.

What it means

Release value early and often. Break work into smaller chunks that can be delivered on short timelines, so teams can gather feedback, make improvements, and demonstrate progress regularly.

The principle in practice

- ✔ **Permit applications:** A city team rebuilds its online building permit service. They deploy the core functionality for residential permits first, then release support for commercial, multi-unit, and heritage properties in iterative cycles, testing and improving as they go.

- ✔ **Youth employment initiatives:** A labour department pilots a job placement program for recent grads in the public sector. The first release includes 30 placements with basic tracking. Later phases add private-sector partners, mentorship support, and participant feedback loops.
- ✔ **Library modernization:** A library tests new self-checkout and digital reservation tools in three locations over six weeks. After gathering feedback and resolving issues, they continue rolling out the updated experience in small batches across the network.

4. Business people and developers must work together daily throughout the project.

What it means

Collaboration is frequent and continuous, not limited to scheduled meetings or handoffs. Decision-makers, policy leads, designers, developers, and delivery teams need to work side by side to stay aligned, unblock each other, and deliver better outcomes.

The principle in practice

- ✔ **Affordable housing program:** Policy, program, legal, data, and procurement teams work in daily syncs during the early phases of a new affordable housing initiative. This tight coordination allows them to iterate on funding models, vendor partnerships, and eligibility without creating bottlenecks.
- ✔ **Civic infrastructure dashboards:** While building a city-wide infrastructure reporting tool, engineers, communications staff, and urban planners meet daily to test language, visuals, and feature priority. This helps ensure the tool works for both the public and internal operations teams.
- ✔ **Transit upgrades:** During a light rail system upgrade, transportation planners, engineers, and comms teams co-locate for the duration of the project. That way, they can address issues like service rerouting or rider confusion in real time, instead of working in silos.
- ✔ **Benefits application platform:** A federal team modernizing a benefits application works in a shared space (virtual or physical) with analysts, service designers, and developers. Daily collaboration allows quick decisions on eligibility logic, user flow, and compliance, avoiding long review cycles or misunderstandings.

5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.

What it means

Empower teams to lead the work. When people have clarity, autonomy, and trust, they take ownership and make better decisions. Leadership's role is to remove barriers, not micromanage.

The principle in practice

- ✓ **Cross-functional delivery teams:** A government department forms a digital service team with developers, designers, and program staff led by a Product Owner. Leadership sets the outcomes and provides resources, but the team owns day-to-day decisions, sprint planning, and backlog prioritization.
- ✓ **Youth engagement programs:** A national youth advisory council is given a mandate and funding to co-design community programs. Instead of being consulted at the end, they're supported as full contributors with decision-making power and access to program teams.
- ✓ **Frontline-driven process change:** A housing authority empowers frontline staff to redesign intake processes for clients. Supported with time and facilitation, the team maps pain points and proposes improvements that leadership then funds and implements.
- ✓ **Internal innovation labs:** A provincial digital team creates a dedicated space where multidisciplinary teams can experiment with service improvements. They're trusted to choose tools, set priorities based on user needs, and test ideas without excessive approvals — leading to faster, more relevant solutions.

6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

What it means

Talk early and often — ideally in real time. Direct, real-time communication is more effective than long email chains or formal documents. Written communication is useful, but direct conversation clears up confusion faster, builds trust, and supports better collaboration.



The principle in practice

- ✓ **Daily stand-ups:** A city's digital services team holds short daily online stand-ups with product managers, developers, designers, and content strategists. These real-time check-ins help unblock work, surface problems early, and avoid miscommunication across roles.
- ✓ **Community partnerships:** A provincial housing program moves away from email chains to direct, recurring meetings with community service providers. These conversations surface critical user needs and lead to better-informed program adjustments.
- ✓ **Public engagement via pop-ups:** Instead of relying solely on surveys or mailers, city planners meet residents in person at libraries, events, or pop-up booths to discuss infrastructure changes. These conversations generate richer insights and help teams better understand lived experiences.

7. Working software is the primary measure of progress.

What it means

Progress isn't measured by documents, plans, or meetings. It's measured by how much real, usable value has been delivered. If it doesn't work for the people using it, it's not done.

The principle in practice

- ✓ **Live service over slide decks:** A team redesigning an online permit application avoids long status reports and instead demos the working product every sprint. Stakeholders can see what's functional, test it, and give immediate feedback — keeping the focus on real progress.
- ✓ **Regulatory services:** A department redesigns its inspection tracking system. The team focuses on getting the core case management feature working for inspectors in the field. When inspectors can use the system in real time without relying on paper forms, that becomes the clearest sign of progress.
- ✓ **Simplified form testing:** An agency redesigns its business registration forms and considers progress made only when real users can complete the form without confusion. Internal signoffs or finished mock-ups don't count. What matters is whether people can use the form successfully.
- ✓ **Transit project phasing:** Instead of reporting on planning milestones, a transit agency launches limited light rail service on a completed portion of the route. Actual performance, rider feedback, and reliability become the true indicators of how the project is progressing.

8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

What it means

Pace and expectations should be steady, realistic, and respectful of the people doing the work. Sustainable teams deliver better results over time.

The principle in practice

- ✓ **Health equity programs:** A provincial health team launches a mental health access program focused on underserved communities. They work closely with community organizations to phase implementation over 18 months, building in rest points. This supports sustained collaboration and avoids burnout for frontline partners.
- ✓ **IT and operations:** An internal IT team sets clear expectations with departments about release schedules. Rather than pushing out features based on political urgency, they follow a predictable roadmap that balances speed with maintainability. This reduces last-minute requests and system instability.

- ✔ **Digital delivery teams:** A provincial digital team moves from crunch-time launches to a cadence of two-week sprints with defined work limits. They stop taking on more than they can realistically deliver. Regular retrospectives help the team manage workloads, reduce stress, and avoid burnout.
- ✔ **Emergency income supports:** During a crisis, a federal agency avoids over-engineering a new income support system. They launch with a simple, repeatable intake model that staff can manage consistently, even under high demand. Later enhancements are spaced out so internal teams can keep up without working overtime.

9. Continuous attention to technical excellence and good design enhances agility.

What it means

Well-structured systems, services, and policies are easier to change. Investing in quality, from clean code to thoughtful service design, makes it faster and safer to adapt when things shift.

The principle in practice

- ✔ **Design systems in government:** A central digital office maintains a government-wide design system. By standardizing components like form fields, buttons, and accessibility guidance, teams across departments can build faster, with less rework, and ensure consistency for users. It also reduces tech debt and makes iteration easier.
- ✔ **Procurement modernization:** A central procurement team builds a modular contract framework with reusable language for common service types. This improves legal clarity, reduces time spent rewriting the same terms, and enables faster procurement for digital projects, especially when requirements evolve during development.
- ✔ **Data infrastructure:** A city invests in high-quality, well-documented data pipelines for transit, housing, and emergency response. Teams can reliably use shared datasets without constantly cleaning or interpreting them, which makes it easier to test new programs, develop dashboards, or improve services without friction.

10. Simplicity — the art of maximizing the amount of work not done — is essential.

What it means

Focus on what truly matters. Don't overbuild, overplan, or overcomplicate. The simplest solution that meets the need is often the best starting point, and it creates space to adapt as you learn.

The principle in practice

- ✓ **Service operations:** A contact centre team reworks its knowledge base by archiving outdated scripts and rewriting top-requested answers in plain language. Removing clutter helps staff find what they need quickly, improves accuracy, and cuts down on call times.
- ✓ **Grant application:** A small team launching a new grant application portal starts with the absolute basics: clear eligibility info, a simple form, and an email confirmation. They hold off on building dashboards or user accounts until there's clear demand. Early delivery gets real feedback fast, without unnecessary features.
- ✓ **Policy implementation:** Instead of introducing an entirely new benefit, a government tweaks an existing tax credit to make it more accessible to low-income households. Using existing systems avoids new infrastructure, training, or communications and delivers impact faster.

11. The best architectures, requirements, and designs emerge from self-organizing teams.

What it means

Great ideas come from the people closest to the work. When teams have the autonomy to make decisions and collaborate across disciplines, they create better solutions because they figured out what works.

The principle in practice

- ✓ **Infrastructure renewal:** A municipal team managing flood mitigation empowers field engineers, data analysts, and local planners to collaborate directly. They design adaptable infrastructure using local knowledge, such as natural flood routes and housing patterns, without top-down interference from central planning.

- ✔ **Education programs:** An education department creates a digital learning initiative but lets school districts shape how it's implemented. Some co-design with local students. Others adapt the tools for bilingual classrooms. The core team supports with coaching, not controls, and collects lessons across teams.
- ✔ **Community safety programs:** Instead of dictating a one-size-fits-all strategy, a public safety department invites regional teams to co-design approaches to mental health crisis response. One team partners with peer support workers, another integrates nurses into call centres. Each model is grounded in local experience.

12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

What it means

Improvement isn't a one-time event. It's a habit. Teams pause regularly to look at what's working, what's not, and what they could do differently. These small, frequent adjustments make a big impact over time.

The principle in practice

- ✔ **Frontline service delivery:** Staff at a local Service Centre meet biweekly to share observations and improvement ideas. One session reveals that walk-in visitors are confused about check-in. The team quickly pilots new signage and tracks a measurable drop in complaints within two weeks.
- ✔ **Emergency management:** After every major response, an emergency coordination team runs structured debriefs across departments. Following a severe storm, the review highlights coordination gaps with external partners. They adjust protocols, test them in the next simulation, and build confidence across agencies.

Key Agile concepts

Agile includes a number of terms that may be unfamiliar at first. These concepts help teams structure their work, collaborate effectively, and stay focused on delivering value. Here's what they mean:

Agile

A way of working that aligns with the Agile Manifesto's values and principles, and prioritizes collaboration, transparency, and responsiveness to change.

Agile Manifesto

A foundational document created in 2001 that outlines the values and principles of Agile.

Backlog

A prioritized list of features, fixes, and tasks that the team needs to complete. The product backlog is a dynamic source of truth, continuously refined and reprioritized as new information emerges.

Cross-functional team

A team made up of people with different skill sets, such as architecture, design, research, development, testing, and operations, who work together to deliver a complete solution.

Daily stand-up (or daily Scrum)

A short daily meeting (usually 15 minutes) where team members share what they're working on, highlight blockers, and coordinate next steps. It helps the team stay aligned and adapt quickly.

Definition of done

A shared understanding of what it means for work to be complete. This might include meeting acceptance criteria, passing tests, getting approvals, or being ready for release.

Developers

Anyone involved in building and delivering a working solution, not just software engineers. In Agile, "developer" refers broadly to the people doing the work needed to turn ideas into outcomes.

Increment

The usable result of a sprint. It's the working solution or output delivered at the end of an iteration, something that can be shown, tested, or used to gather feedback.

Iteration

Another word for a sprint. Agile teams often work in short iterations to test ideas, learn quickly, and continuously improve their solutions.

Kanban

A visual way of managing work as it moves through a process. It uses columns (e.g., To Do, In Progress, Done) to make work visible, limit work in progress, and identify blockers.

Retrospective

A regular opportunity for the team to reflect on how they're working and identify ways to improve. It's focused on learning, not blame.

Self-organizing team

A team that manages its own work. Instead of being told exactly how to do things, team members decide together how best to approach the work and meet their goals.

Sprint

A short, focused period (usually one to four weeks) where the team works to deliver a small, valuable piece of work. Each sprint begins with planning and ends with a review.

Sprint planning

A collaborative meeting at the start of each sprint where the team decides what to work on and how they'll do it.

Sprint review

A session at the end of the sprint where the team shows what they've completed, gathers feedback, and discusses what to do next.

Time-boxed

A set amount of time allocated for an activity or decision. Time-boxing helps teams stay focused, make progress, and avoid overthinking or overbuilding.

User story

A brief description of a user's need, written from their perspective. It helps the team understand what someone wants to do and why. Example: "As a parent, I want to apply for childcare support online so I don't have to visit an office."

Popular Agile frameworks and approaches

Agile is not a one-size-fits-all approach. Instead of following a rigid methodology, teams can choose the practices and frameworks that best support their work. They might adopt a single framework, adapt one to fit their context, or combine elements from several approaches to create something that works for them.

What matters most is focusing on Agile values and principles, rather than being dogmatic about specific processes. Agile is not about following a checklist — it's about working in ways that support collaboration, adaptability, and continuous improvement.

There are many Agile frameworks to choose from, each offering different structures and strengths. Teams can explore these options and draw from the ones that best align with their size, composition, and the type and complexity of their work.

Scrum

Scrum is the most widely adopted Agile framework, created by Ken Schwaber and Jeff Sutherland in the early 1990s. It's grounded in empirical process control, working in short, transparent cycles that allow teams to inspect progress and adapt their approach. Scrum promotes iterative development, cross-functional collaboration, and continuous improvement through structured events and defined roles. It is built around the idea of delivering usable solutions in small increments, called sprints, and regularly involving stakeholders for feedback.

When to use it

Scrum is a good fit when stakeholder involvement and team collaboration are critical to success. It is a good choice for cross-functional teams that can commit to focused, time-boxed iterations and who need opportunities to incorporate data, feedback and reassess direction.

How it works

Scrum organizes work into short, fixed-length cycles called sprints that are usually two to four weeks long. At the start of each sprint, the team selects a set of high-priority items from a shared backlog and defines a clear sprint goal. These items represent the most valuable work to deliver based on current needs and feedback.

The team collaborates daily to track progress, adapt to change, and remove blockers. At the end of the sprint, they share what they've built with stakeholders to gather feedback, then hold a retrospective to reflect on how they worked and identify ways to improve.

Scrum provides structure through clearly defined events and roles, but it's flexible in how teams apply them. The goal is to support focus, accountability, and continuous learning while maintaining a steady pace of delivery.

Core practices

- ✔ Define clear roles: one to prioritize (Product Owner), one to support the process (Scrum Master), and a cross-functional team to deliver the work
- ✔ Maintain a prioritized backlog that evolves with new insights and feedback
- ✔ Plan the sprint by selecting top-priority work and setting a clear goal
- ✔ Hold daily check-ins to align progress, surface blockers, and adapt as needed
- ✔ Involve stakeholders throughout to stay aligned with user and organizational needs
- ✔ Deliver working solutions at the end of each sprint for feedback and validation
- ✔ Reflect as a team after each sprint to identify improvements and adjust accordingly

Roles

- **Product Owner** – prioritizes work and defines goals
- **Scrum Master** – facilitates the process and removes blockers
- **Developers** – the collaborative, cross-functional team that delivers the work

Strengths

- Encourages early and frequent delivery of value
- Promotes stakeholder engagement and transparency
- Supports continuous learning and adaptation
- Well-suited for complex digital projects with evolving needs

Weaknesses

- Can feel rigid if teams treat Scrum events as checklists
- Requires time commitment and discipline
- Assumes a stable, dedicated team

Learn more

- [Get the official Scrum Guide](#)



Kanban

Kanban was adapted from lean manufacturing practices at Toyota and introduced to knowledge work by David J. Anderson in the early 2000s. It focuses on managing flow rather than fixed iterations. Kanban emphasizes visualizing work, limiting work in progress (WIP), and making incremental changes to improve efficiency and delivery. Unlike Scrum, Kanban does not prescribe specific roles or ceremonies, making it lightweight and easy to adopt without restructuring teams.

When to use it

Kanban works well for teams managing a continuous flow of work with shifting priorities. It's a strong choice when tasks vary in size and urgency, and the team needs flexibility to adapt without the structure of fixed sprints. Kanban is especially useful when visibility, flow efficiency, and workload management are more important than iterative planning.

How it works

Kanban is a flow-based approach that helps teams manage work as it moves through stages, from start to finish. Instead of working in fixed-length cycles, teams focus on visualizing all work in progress and continuously delivering value as items are completed.

Work is pulled from a prioritized backlog or intake queue and moved across a board with columns that reflect the team's workflow such as "To Do," "In Progress," and "Done."

Unlike Scrum, Kanban doesn't define roles or prescribe time-boxed events. Teams can adopt it incrementally without restructuring how they work. The emphasis is on improving how work flows through the system, identifying blockers early, and limiting how much is in progress at any given time to prevent overload.

Kanban provides flexibility and visibility while promoting a culture of continuous improvement. Over time, teams use metrics like cycle time and throughput to refine how they plan, prioritize, and deliver work.

Kanban includes a set of core practices:

- ✓ Visualize all work on a shared board to improve transparency and alignment
- ✓ Limit work in progress to reduce context-switching and avoid bottlenecks
- ✓ Pull new work only when capacity is available, rather than assigning in advance
- ✓ Track how long tasks take to move through the system and use this data to improve
- ✓ Make workflow policies visible so expectations are clear and consistent
- ✓ Review the board regularly to spot issues and adjust in real time
- ✓ Focus on continuous delivery rather than fixed timelines or batch releases

Roles

Kanban has no required roles. Teams continue using their current roles and processes while adopting Kanban practices to improve flow and visibility.

Strengths

- Easy to implement and adaptable to existing processes
- Supports continuous delivery and responsiveness to change
- Improves visibility, transparency, and accountability
- Helps identify bottlenecks and reduce inefficiencies

Weaknesses

- Less built-in structure for planning and reflection
- Can lead to unmanaged work if WIP limits aren't enforced
- Less effective for large or cross-functional initiatives without added coordination

Extreme Programming (XP)

Extreme Programming (XP) is an Agile framework created by Kent Beck in the late 1990s to help software teams deliver high-quality code in fast-changing environments. XP focuses on engineering excellence, tight feedback loops, and frequent releases. It introduces a set of specific technical practices, such as pair programming, test-driven development, and continuous integration, to improve code quality, reduce defects, and support sustainable development. XP was originally designed for software development but many of its principles apply broadly to any technical delivery team.

When to use it

XP is a good fit for development teams building complex systems in fast-changing environments. It's most effective when technical quality is critical, requirements are expected to evolve, and the team can work closely with a dedicated customer. XP is ideal when rapid feedback, strong engineering practices, and shared ownership are needed to reduce risk and deliver stable, maintainable solutions.

XP requires strong team cohesion and technical discipline. It's often used in combination with other frameworks like Scrum or Kanban to cover broader delivery processes.

How it works

Extreme Programming is built around short, consistent iterations, usually one to two weeks long. At the start of each cycle, the team works closely with a customer to prioritize user stories and define what success looks like. These stories guide development and are paired with tests to validate the outcomes.

Developers work in pairs, write tests before writing code, and integrate their work frequently to keep the product in a working state. The entire team shares ownership of the codebase, encouraging collaboration and continuous improvement. Testing and feedback are built into the process, helping the team adapt quickly and deliver high-quality solutions without burning out.

Core practices

- ✓ Write automated tests before writing any code to ensure functionality from the start
- ✓ Develop code in pairs to share knowledge and solve problems together
- ✓ Integrate changes frequently to catch issues early and maintain stability
- ✓ Improve code structure continuously without changing how it behaves
- ✓ Release small, working updates often to reduce risk and gather feedback
- ✓ Allow any developer to improve any part of the codebase at any time
- ✓ Maintain a consistent, sustainable pace to support long-term productivity

Roles

- **Customer** – sets priorities and provides feedback
- **Developer** – writes and tests code collaboratively
- **Tester** – defines and runs acceptance tests
- **Tracker** – monitors progress and team metrics (optional)
- **Coach** – guides the team in XP practices

Strengths

- Improves code quality and reduces technical debt
- Enables rapid response to changing requirements
- Promotes collaboration and shared ownership of solutions
- Provides built-in mechanisms for continuous improvement

Weaknesses

- Less suitable for non-technical teams
- Requires high team maturity and discipline to sustain technical practices
- Pair programming and TDD can be resource-intensive if not well supported
- Challenging in environments where continuous integration infrastructure is limited

Other Agile frameworks

In addition to Scrum, Kanban and XP, there are other Agile methodologies and approaches that may be useful depending on your team's needs. This is not an exhaustive list:

SAFe (Scaled Agile Framework)

A structured approach for applying Agile at enterprise scale. Commonly used in large organizations with multiple Agile teams, SAFe provides alignment, collaboration, and delivery across teams.

Scrum@Scale

Extends Scrum practices beyond a single team to coordinate multiple teams working together on complex, interconnected projects. It enables organizations to scale without losing the core principles of Scrum.

Feature-Driven Development (FDD)

Focuses on delivering tangible, working software repeatedly through short, client-valued features. Ideal for projects where functionality can be broken into small, discrete pieces.

Crystal Agile Methodology

A family of Agile methodologies that prioritize people, interactions, and team dynamics over rigid processes. Different versions (like Crystal Clear or Crystal Orange) are tailored to team size and project criticality.

Agile Marketing

Adapts Agile principles to marketing workflows, emphasizing iterative campaign development, customer feedback, and rapid response to change. Often uses sprints, stand-ups, and Kanban boards.

Agile HR

Applies Agile values to human resources practices, focusing on collaboration, responsiveness, and transparency. Encourages cross-functional HR teams and iterative improvement, often championed by the Agile HR Community.

eduScrum

A collaborative educational approach where students work in self-directed teams on assignments or projects. Encourages ownership, responsibility, and continuous learning in a structured, Scrum-like environment.

Agile Classrooms

An educational framework that makes learning more adaptive, student-centered, and iterative. Incorporates Agile tools and mindsets to empower students and teachers to co-create the learning experience.

The Digital Transformation Team's Human-Centered Agile Framework

Human-Centered Agile is the fusion framework developed by the Government of New Brunswick's Digital Transformation Team. It brings together the structure of Scrum, the visibility of Kanban and the principles of human-centered design to help teams explore problems deeply, deliver iteratively, and stay focused on real-world impact.

This approach is designed for multidisciplinary teams working in complex public sector environments, where services must meet diverse user needs, align with policy and operational realities, and adapt to ongoing change. Human-Centered Agile makes space for both research and delivery — helping teams avoid jumping to solutions too quickly, while maintaining momentum and accountability.

By integrating discovery, design, and delivery into a single end-to-end process, this framework supports more thoughtful decisions, better alignment across roles, and faster learning through continuous feedback. The goal is not just to ship usable solutions, but to deliver real value, with measurable improvements for the people and communities we serve.

When to use it

Human-Centered Agile is a good fit when you need to understand a complex problem space before moving into delivery. It works well for teams that want to balance research and iteration, and when cross-functional collaboration is critical to success.

It's especially useful when you're designing or improving digital services that impact people directly, where user needs, policy requirements, and technical considerations all need to be considered together. This framework helps teams build shared understanding, reduce risk, and stay aligned on outcomes as they work through ambiguity.

You might consider using this approach when:

- You need to explore the problem space before committing to solutions
- The solution requires input from users, business owners, and technical teams
- Success relies on collaboration between design, development, and business roles
- You want to reduce risk and build trust through early and continuous delivery
- You're working in an environment that values iteration, collaboration, and learning

How it works

Human-Centered Agile combines the iterative structure of Scrum with the visibility of Kanban and the discovery-driven mindset of human-centered design. Work is organized into streams that blend research, design, development and delivery — enabling teams to learn early, adapt quickly, and deliver valuable outcomes.

Each initiative begins with discovery to understand the people, systems, and policies involved. Insights are translated into a shared vision, a prioritized backlog, and a clear problem definition. From there, teams work in short sprints to design, develop, and test solutions, gathering feedback along the way.

The framework encourages constant collaboration across roles and with stakeholders. Agile ceremonies like sprint planning, daily stand-ups, and retrospectives provide structure, while flexible design and research methods keep the team aligned with real-world needs.

Core practices

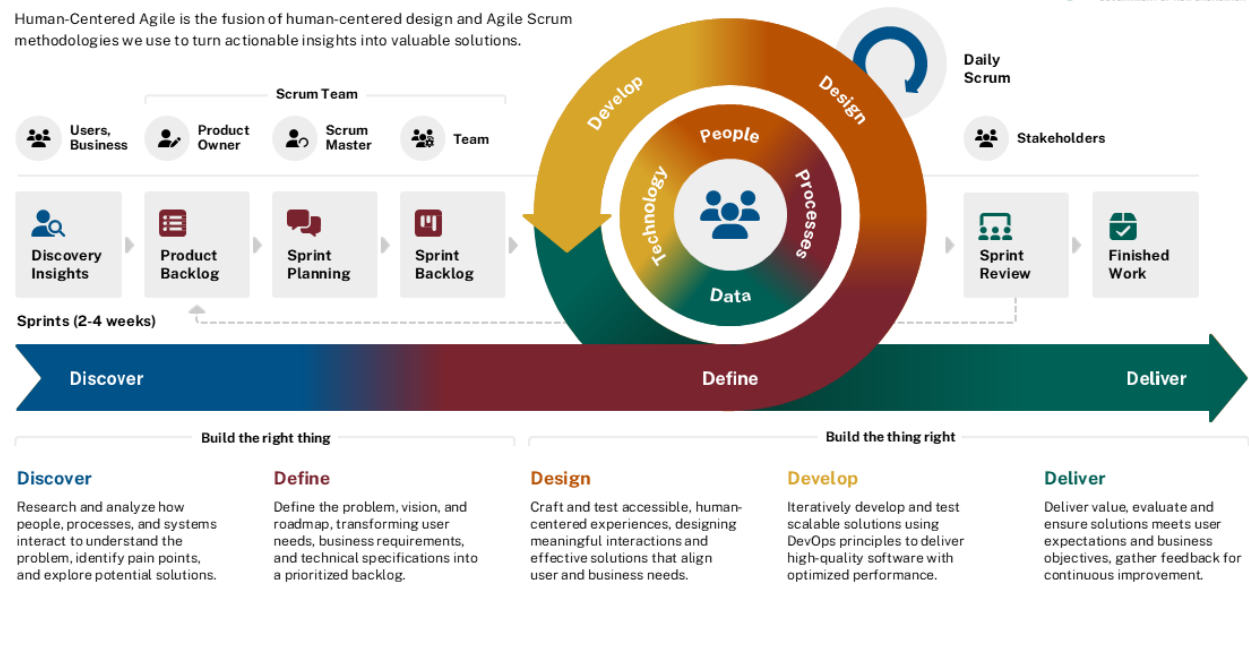
Human-Centered Agile is structured around short, focused sprints that combine research, design, and delivery. Each sprint typically lasts one to two weeks and begins with sprint planning. Progress is made visible through daily stand-ups, and teams reflect through sprint reviews and retrospectives as needed. These agile rhythms support focus, alignment, and adaptability.

The five core practices outlined below help teams move from understanding problems to delivering real-world impact — while staying grounded in user needs, policy realities, and technical feasibility:

- ✓ **Discover:** Research and analyze how people, processes, and systems interact to understand the problem, identify pain points, and explore potential solutions.
- ✓ **Define:** Define the problem, vision, and roadmap, transforming user needs, business requirements, and technical specifications into a prioritized backlog.
- ✓ **Design:** Craft and test accessible, human-centered experiences, designing meaningful interactions and effective solutions that align user and business needs.
- ✓ **Develop:** Iteratively develop and test scalable solutions using DevOps principles to deliver high-quality software with optimized performance.
- ✓ **Deliver:** Deliver value, evaluate and ensure solutions meets user expectations and business objectives, gather feedback for continuous improvement.

Human-Centered Agile

Human-Centered Agile is the fusion of human-centered design and Agile Scrum methodologies we use to turn actionable insights into valuable solutions.



Roles

The framework depends on a cross-functional team, where each member contributes specific expertise to deliver high-quality, user-centered solutions. Primary roles reflect the functions people play within the team, not their job titles.

Members are encouraged to explore activities beyond their primary roles to support learning, collaboration, and shared ownership. Open communication across all roles is essential to support effective decision-making and build strong, cohesive teams.

- **Product Owner**
Defines the product vision, prioritizes work, and ensures alignment between user needs, policy goals, and technical delivery.
- **Scrum Master**
Facilitates team collaboration, removes blockers, and supports continuous improvement through agile practices.
- **Systems Architect**
Designs scalable, secure, and maintainable technical solutions that align with enterprise architecture and long-term strategy.

- **Business Analyst**
Identifies business and user needs, maps processes, and defines functional requirements in collaboration with stakeholders.
- **Business Systems Analyst**
Translates business requirements into technical specifications, evaluates system impacts, and supports integration planning.
- **UI/UX Designers**
Designs user-centered experiences based on research, usability standards, and iterative testing.
- **Developers**
Builds, tests, and delivers working software in short, iterative cycles, collaborating closely with designers and analysts.
- **Adoption and Training Leads**
Ensures the solution is effectively adopted through change management, user training, and ongoing support strategies.

Strengths

- Aligns user, business, and technical perspectives
- Enables rapid learning and iteration
- Builds momentum and trust through early delivery
- Encourages shared ownership and team growth
- Adapts to changing project and organizational needs

Weaknesses

- Requires cultural and mindset shifts across disciplines
- Demands strong facilitation and cross-role communication
- Can be resource-intensive during discovery and definition phases
- Success depends on continuous stakeholder engagement

Get started with Agile

Agile isn't something you adopt all at once. It's a shift in mindset, behavior, and team culture. The best way to begin is by starting small — learning by doing, reflecting often, and adjusting based on what works in your context. Here are five steps to get started:

1. Start with your team

Bring your team together and talk about how you work today. What slows you down? Where do handoffs create confusion? What could help you collaborate better? Pick one or two practices to try:

- ✓ A daily stand-up to stay aligned
- ✓ A shared board to make work visible
- ✓ A short sprint to test something and reflect

Start small and treat it as an experiment. Focus on building shared understanding, not following a process perfectly.

2. Choose a project you can learn from

Look for a project where your team can test Agile practices in real conditions. It doesn't need to be high-stakes or high-profile. It should be meaningful, manageable, and open to iteration. A good starting point is a project where:

- ✓ The team has space to experiment
- ✓ User feedback can be gathered quickly
- ✓ Delivery can happen in stages, not all at once

Treat the work as a learning opportunity. Pay attention to how the team works together, not just what gets delivered.

3. Make your work visible

When work is visible, it's easier to collaborate, spot blockers, and manage expectations. Even a simple board with columns like "To do," "In progress," and "Done" can improve transparency and coordination. You can start by:

- ✓ Using sticky notes or a digital board
- ✓ Reviewing progress weekly with your team
- ✓ Updating it often so it reflects what's really happening

This visibility helps build trust and shared ownership — within the team and with stakeholders.

4. Focus on outcomes, not output

Agile teams stay focused on the impact of their work. What problem are you trying to solve? Who is it for? How will you know it's working?

Start by shifting conversations from tasks or deliverables to outcomes. Describe what success looks like for the people or communities you're serving. Check in regularly to ensure the work remains aligned with that purpose.

5. Learn by observing what works — and what doesn't

Agile is not about following a single framework. It's about finding ways of working that help your team collaborate, adapt, and deliver value. As you try different practices, pay attention to what's helping and what's getting in the way.

Over time, you may find that a specific framework like Scrum or Kanban fits your context. Or you may need to adapt elements — or build your own approach that reflects your team's needs, goals, and constraints. What matters most is that your way of working supports continuous learning and stays focused on real outcomes. Change doesn't need to be big to be meaningful.

Start small, stay curious, and continuously improve.